

HANDS-ON WORKSHOP

Build with Codex + GitHub

Ship one interactive widget — using only AI prompts.



THE GOAL

One engineer. One widget. One pull request.

1

shared live page —
the Widget Wall

30

engineers, each owning
one widget

0

lines of code you
write by hand

By the end you'll have connected Codex, received a task, built a working widget, and merged it to the live wall.

GitHub connector vs. Codex

R

GitHub connector

- Reads and explains your repo
- Answers questions, cites code
- Cannot commit, branch, or open PRs

AI

Codex (the agent)

- Edits code in a sandbox
- Runs tests, commits, pushes
- **Opens the pull request**

You need BOTH: connector enabled for repo Q&A, Codex for the actual work.

From branch to live wall

1 Create your branch

workshop/<legal-id> — this fires your task

2 Get your task

a GitHub Action opens an Issue with your widget spec

3 Build with Codex

prompt it to write your widget file

4 Open a pull request

the check loads your widget in a headless browser

5 Merge to main

facilitator reviews and merges

6 Auto rebuild + deploy

the wall redeploys to GitHub Pages

Your interactive widget



widgets/pagduldx.js

```
WORKSHOP.register({  
  id: "pagduldx",  
  title: "Click Counter",  
  author: "pagduldx",  
  mount: function (root) {  
    // build DOM in root,  
    // wire up the behavior  
  },  
});
```

The shared wall

The page auto-discovers your file and mounts it as its own card. It starts empty and fills as PRs merge.

Your id must equal your file name — both are your legal ID.

Three things to have ready



Your own ChatGPT with Codex

Confirm you can see Codex in the left nav — the agent that opens PRs.



The shared GitHub credentials

Everyone connects Codex to the same repo with the credentials provided.



Your legal ID

e.g. pagduldx — used for your branch and your widget file name.

You install nothing — just git and Codex. GitHub runs the tests for you on every pull request.

STEP 1 — CONNECT

1

Connect GitHub, open Codex



verification prompt

```
# ChatGPT: Settings -> Apps -> GitHub -> authorize, open Codex
```

Confirm you can access the workshop repository.

Read README.md and examples/example-widget.js and tell me:

1. what the page does and how widgets appear on it
2. the exact object a widget file must register
3. which files I must not edit.

Do not modify anything.

Full prompts are in the Participant Guide.

STEP 2 — BRANCH

2

Create your branch



what to create

```
# GitHub web UI -> branch dropdown -> type name -> create from main
```

```
workshop/<your-legal-id>
```

```
# Pushing this branch fires a GitHub Action that opens  
# your widget task Issue. Wait about a minute.
```

Full prompts are in the Participant Guide.

STEP 3 — YOUR TASK

3

Read your assigned Issue



prompt to Codex

```
# Open Issues -> find the one with your legal ID
```

```
Find the open assignment Issue whose title  
contains my legal ID. Summarize:
```

- which widget I must build and what it should do
- the id / title / mount contract
- which files I must not edit.

```
Do not modify any files.
```

Full prompts are in the Participant Guide.

STEP 4 — BUILD

4

Build your widget with Codex



prompt to Codex

```
Build my assigned widget on branch workshop/<legal-id>.
- Create only widgets/<legal-id>.js, mirroring the example template
- Register one widget whose id equals "<legal-id>"
- Build all DOM inside root; wire it so it actually works
- Self-contained: no external libraries, no network calls
- Do NOT edit index.html, loader.js, styles.css, registry.js
Summarize the file you created. Do not commit yet.
```

Full prompts are in the Participant Guide.

Review the diff



prompt to Codex

```
Show the full diff for my branch and confirm:  
1. only widgets/<legal-id>.js was added  
2. no shared file changed (index.html, loader.js,  
   styles.css, registry.js)  
3. no other engineer's widget changed  
4. it registers with id = my legal ID and mounts  
Do not commit yet.
```

Full prompts are in the Participant Guide.

STEP 6 — COMMIT

6

Commit and push



prompt to Codex

```
# You install nothing locally – the PR check validates  
# your widget on GitHub in the next step.
```

```
Commit only widgets/<legal-id>.js with message:
```

```
"<TASK-ID>: add <legal-id> widget"
```

```
Push workshop/<legal-id>. Do not push to main.
```

```
Do not force-push.
```

Full prompts are in the Participant Guide.

STEP 7 — PULL REQUEST

7

Open the PR, let the check run



prompt to Codex

Open a PR from workshop/<legal-id> into main.

Title: "<TASK-ID>: add /<legal-id> widget"

Reference my Issue with "Closes #<n>".

Do not merge. Return the PR number.

The PR check loads your widget headlessly and

confirms it registers and works. Wait for green.

If the check goes red: copy the log, paste it to Codex, and push again.

8

Merge, rebuild, live — automatically



Merge

The facilitator reviews your green PR and merges it.



Rebuild + deploy

GitHub Actions regenerates the wall and redeploys it to Pages.



See it live

Refresh the URL — your card is now on the wall with everyone's.

Merges happen in batches — a burst of deploys collapses to the latest one.

The contract & the rules

The widget contract

- id must equal your file name
- Build everything inside root
- Self-contained — no libraries, no network

Rules

- One widget per file, named by your legal ID
- **Never edit `index.html`, `loader.js`, `styles.css`, `registry.js`, or others' widgets**
- No credentials or secrets anywhere
- Read the PR check; fix red before merge

LET'S BUILD

Create your branch to get started.



branch from main

```
workshop/<your-legal-id>
```

Guides in the repo: [PARTICIPANT_GUIDE.md](#) · [PROMPTING_GUIDE.md](#)